Informe de Laboratorio 02

Tema: JavaScript en el Cliente

Nota

Estudiante	Escuela	Asignatura
Gustavo Linares Aquino glinares@unsa.edu.pe Geisel Reymar Pacheco Medina gpachecome@unsa.edu.pe Jesus Francisco Silva Pino jsilva@unsa.edu.pe Semestre: I Código: 2025000	Escuela Profesional de Ingeniería de Sistemas	Programación

Laboratorio	Tema	Duración
02	JavaScript en el Cliente	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2025 - A	Del 22 Abril 2026	Al 23 Abril 2026

Índice

1. Tarea **3**

2. Desarrollo **3**
2.1. Plantilla Base HTML 3

3. Estructura del Proyecto **3**

4. Ejercicio 01: Generación de teclado dinámico **4**
4.1. Descripción del Ejercicio 4
4.2. Estructura del Proyecto y HTML 4
4.3. Implementación en JavaScript (Modo Desarrollo) 4
4.4. Lógica de Aleatoriedad y Ofuscación 7
4.5. Pruebas y Resultados de Ejecución 8

5. Ejercicio 02: Calculadora con Historial (Pila) **9**
5.1. Descripción de la Tarea . . . 9
5.2. Análisis de la Solución Técnica . . . 9
5.3. Implementación del Código (Modo Desarrollo) . . . 10
5.4. Protección y Producción (Ofuscación) . . . 11
5.5. Resultados de Ejecución . . . 12

6. Ejercicio 03: Juego del Ahorcado con Canvas **14**
6.1. Descripción de la Tarea . . . 14
6.2. Construcción Dinámica de la Interfaz . . . 14
6.3. Lógica del Juego y Gestión de Estados . . . 15
6.4. Renderizado Progresivo con Canvas . . . 15
6.5. Resultados de Ejecución . . . 16
6.6. Versión de Producción . . . 16

7. Ofuscación del Código **17**

8. Despliegue con Docker **17**
8.1. Dockerfile . . . 17

9. Resultados **17**

10. Cuestionario **18**

11. Conclusiones **18**

12. Repositorio del Proyecto **18**

13. Video **18**

14. Bibliografía **18**

15. Rúbricas **18**
15.1. Entregable Informe . . . 18
15.2. Rúbrica para el contenido del Informe y demostración . . . 19

1. Tarea

Implementar ejercicios utilizando JavaScript nativo ejecutado en el lado del cliente. Toda la interfaz debía generarse dinámicamente mediante JavaScript, sin utilizar etiquetas HTML visibles dentro del cuerpo del documento.

También se solicitó automatizar el despliegue mediante Docker, publicando el laboratorio en la ruta:

`/lab02`

Requisitos principales:

- Uso de JavaScript nativo.
- Generación total del DOM.
- Código en desarrollo y producción.
- Ofuscación del código JavaScript.
- Despliegue mediante Docker.

2. Desarrollo

2.1. Plantilla Base HTML

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Laboratorio 02</title>
</head>
<body>
<!-- no html, generate it with javascript -->
<script src="ejercicio01.min.js"></script>
</body>
</html>
```

3. Estructura del Proyecto

El proyecto fue organizado en carpetas para separar código fuente, producción y despliegue.

```
Grupo-7---Laboratorio/
|-- lab02/
|   |-- ejercicio01.html
|   |-- ejercicio02.html
|   |-- js/dev/
|   |-- js/prod/
|-- Dockerfile
|-- README.md
```

4. Ejercicio 01: Generación de teclado dinámico

4.1. Descripción del Ejercicio

En este ejercicio se implementó un teclado numérico utilizando JavaScript nativo. Toda la interfaz fue generada dinámicamente mediante la manipulación del *Document Object Model* (DOM), cumpliendo con el requerimiento de no utilizar etiquetas HTML estáticas en el cuerpo del documento.

4.2. Estructura del Proyecto y HTML

Como se observa en la **Figura 1**, el archivo HTML base (`ejercicio01.html`) tiene una estructura mínima, cuya función principal es servir de contenedor y enlazar el script de JavaScript encargado de la generación de los elementos.

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Ejercicio 01</title>
</head>
<body>
<script src="js/prod/ejercicio01.js"></script>
</body>
</html>
```

Figura 1: Archivo HTML del ejercicio 01 (Solo enlace a script).

4.3. Implementación en JavaScript (Modo Desarrollo)

La implementación se basó en el uso intensivo de `document.createElement()` para definir la estructura del teclado (contenedores, filas y botones) y el método `appendChild()` para su inserción jerárquica.

A continuación, se presentan las capturas del código fuente en su etapa de desarrollo, donde se detalla la configuración de estilos y la lógica de selección de la tarjeta:

ejercicio01.js

js > prod > ejercicio01.js > ...

```javascript
const texto2 = document.createElement("span");
texto2.innerText = "Número de tarjeta:";
texto2.style.display = "inline-block";
texto2.style.width = "160px";
texto2.style.textAlign = "right";
texto2.style.marginRight = "10px";

const inputTarjeta = document.createElement("input");
inputTarjeta.type = "text";
inputTarjeta.value = "4214 ";
inputTarjeta.style.width = "220px";
inputTarjeta.style.height = "26px";

inputTarjeta.oninput = function () {
    let texto = inputTarjeta.value;
    let soloNumeros = "";

    for (let i = 0; i < texto.length; i++) {
        let c = texto[i];
        if (c >= "0" && c <= "9") {
            soloNumeros = soloNumeros + c;
        }
    }

    if (soloNumeros.length < 4) {
        soloNumeros = "4214";
    }

    if (soloNumeros.substring(0, 4) != "4214") {
        soloNumeros = "4214" + soloNumeros.substring(4);
    }
```

Figura 2: Creación dinámica de contenedores y elementos de interfaz.

```javascript
fila1.appendChild(selectTarjeta);

const fila2 = document.createElement("div");
fila2.style.marginBottom = "15px";

const texto2 = document.createElement("span");
texto2.innerText = "Número de tarjeta:";
texto2.style.display = "inline-block";
texto2.style.width = "160px";
texto2.style.textAlign = "right";
texto2.style.marginRight = "10px";

const inputTarjeta = document.createElement("input");
inputTarjeta.type = "text";
inputTarjeta.value = "4214 ";
inputTarjeta.style.width = "220px";
inputTarjeta.style.height = "26px";

inputTarjeta.oninput = function () {
    let texto = inputTarjeta.value;
    let soloNumeros = "";

    for (let i = 0; i < texto.length; i++) {
        let c = texto[i];
        if (c >= "0" && c <= "9") {
            soloNumeros = soloNumeros + c;
        }
    }

    if (soloNumeros.length < 4) {
        soloNumeros = "4214";
    }
```

Figura 3: Lógica para la validación y manejo de eventos del teclado.

```
63
64      if (soloNumeros.substring(0, 4) != "4214") {
65          soloNumeros = "4214" + soloNumeros.substring(4);
66      }
67
68      if (soloNumeros.length > 12) {
69          soloNumeros = soloNumeros.substring(0, 12);
70      }
71
72      let resultado = "";
73
74      for (let i = 0; i < soloNumeros.length; i++) {
75          resultado = resultado + soloNumeros[i];
76          if ((i + 1) % 4 == 0 && i != soloNumeros.length - 1) {
77              resultado = resultado + " ";
78          }
79      }
80
81      inputTarjeta.value = resultado;
82  };
83
84  fila2.appendChild(texto2);
85  fila2.appendChild(inputTarjeta);
86
87  const fila3 = document.createElement("div");
88  fila3.style.marginBottom = "15px";
89
90  const texto3 = document.createElement("span");
91  texto3.innerText = "Tipo y Nº Documento:";
92  texto3.style.display = "inline-block";
93  texto3.style.width = "160px";
```

Figura 4: Continuación de la lógica de inserción de elementos en el DOM.

4.4. Lógica de Aleatoriedad y Ofuscación

Un requisito clave fue la implementación de un orden aleatorio para los números del teclado en cada carga de la página. Para la versión de distribución, el código fue sometido a un proceso de ofuscación con el objetivo de proteger la lógica del programa y dificultar su lectura directa en el cliente.

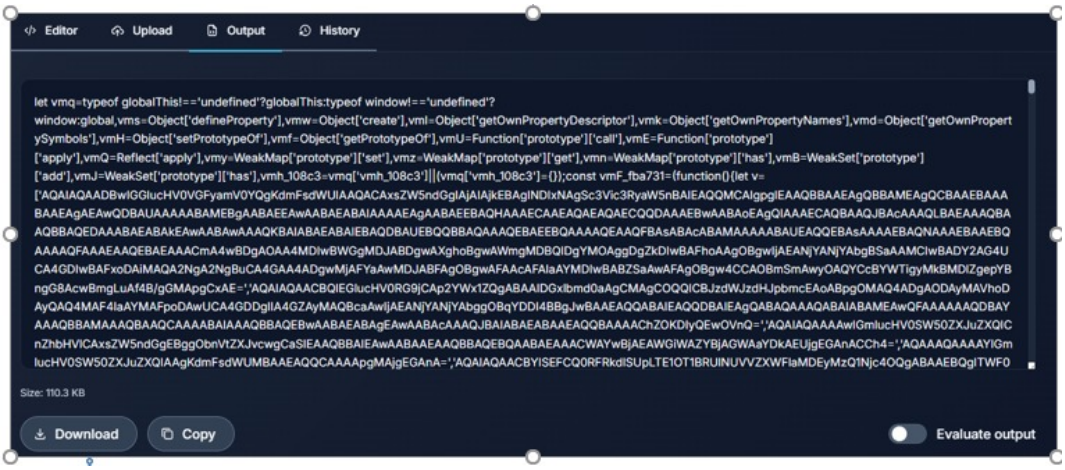

Figura 5: Código JavaScript en modo producción (Versión ofuscada).

4.5. Pruebas y Resultados de Ejecución

Se verificó el funcionamiento del teclado en el navegador, comprobando que la cuadrícula numérica se genera correctamente y cambia su disposición tras cada actualización de la página.

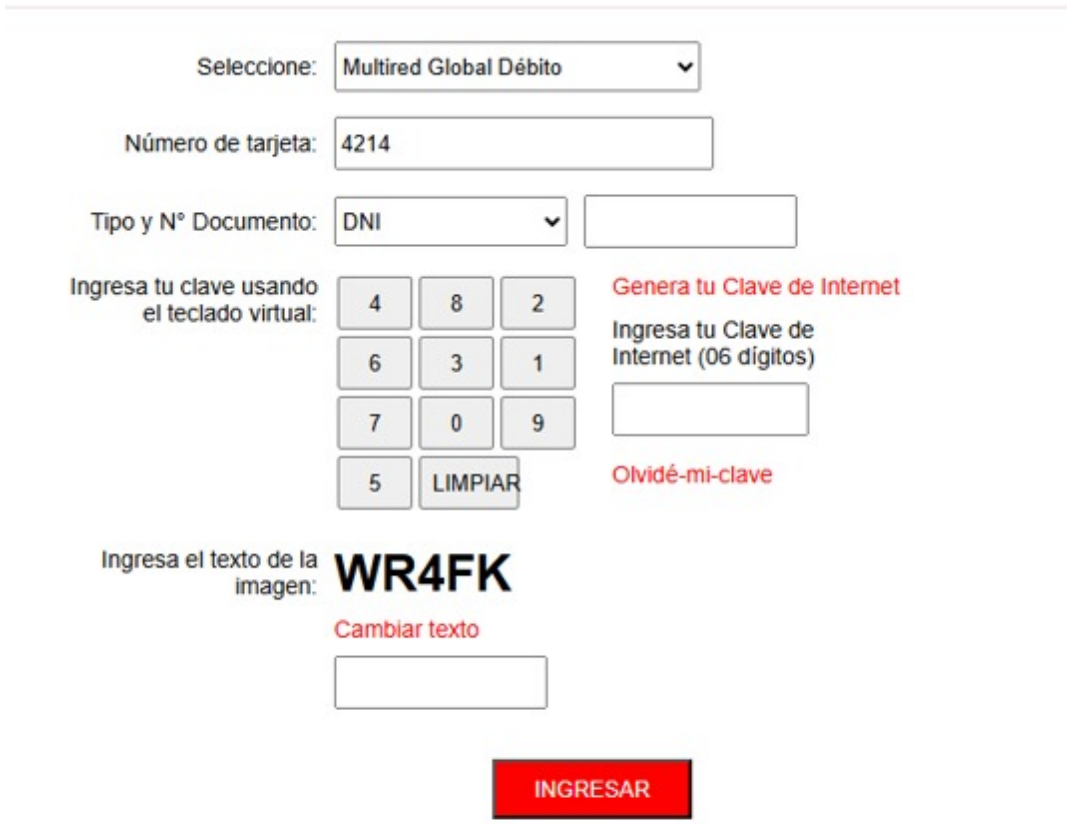

Figura 6: Interfaz del teclado dinámico: Se observa el cambio aleatorio en la posición de los números.

5. Ejercicio 02: Calculadora con Historial (Pila)

5.1. Descripción de la Tarea

El ejercicio consistió en desarrollar una calculadora funcional ejecutada en el cliente, cuyo núcleo de procesamiento utiliza la función `eval()`. Los requisitos técnicos principales incluyeron la generación total de la interfaz mediante la manipulación del DOM y la implementación de una estructura de datos tipo Pila (Stack) para gestionar el historial de operaciones.

5.2. Análisis de la Solución Técnica

La solución se basa en el patrón de diseño de una *Single Page Application* simplificada, donde JavaScript toma el control total del renderizado.

- **Renderizado Dinámico:** Se utilizaron los métodos `document.createElement()` y `appendChild()` para construir la cuadrícula de botones y el visualizador de forma programática, manteniendo el cuerpo HTML vacío.

- **Evaluación de Expresiones:** Se emplea la función `eval()` para procesar la cadena acumulada de caracteres, lo que permite resolver operaciones matemáticas complejas con una sola llamada de función.
- **Gestión de la Pila (Historial):** El historial se implementó bajo el principio LIFO (*Last-In, First-Out*). Cada operación exitosa se almacena en un arreglo dinámico que se visualiza de forma inversa al pie de la aplicación.

5.3. Implementación del Código (Modo Desarrollo)

A continuación, se presentan fragmentos clave del código fuente que ilustran la lógica de la pila y la evaluación de resultados:

Listing 1: Lógica del historial y evaluación

```
// Fragmento representativo del ejercicio02.js
let pilaHistorial = [];

function calcular() {
    try {
        const resultado = eval(operacionActual);
        const registro = `${operacionActual} = ${resultado}`;

        // Implementacion de la Pila (LIFO)
        pilaHistorial.push(registro);
        actualizarPilaVisual();

        operacionActual = resultado.toString();
        actualizarDisplay();
    } catch (error) {
        alert("Expresion invalida");
    }
}
```

```javascript
// 3. Lógica y Creación de Botones
buttons.forEach(text => {
    const btn = document.createElement('button');
    btn.textContent = text;

    if (text === '=') {
        btn.className = 'equal';
        btn.onclick = () => {
            try {
                const resultado = eval(operacionActual);
                const registro = `${operacionActual} = ${resultado}`;

                pilaHistorial.push(registro);
                actualizarPilaVisual();

                operacionActual = resultado.toString();
                display.value = operacionActual;
            } catch (e) {
                display.value = "Error";
                operacionActual = "";
            }
        };
    } else if (text === 'C') {
        btn.className = 'clear';
        btn.onclick = () => {
            operacionActual = "";
            display.value = "";
        };
    } else {
        btn.className = isNaN(text) && text !== '.' ? 'op' : 'num';
        btn.onclick = () => {
            operacionActual += text;
            display.value = operacionActual;
        };
    }
    btnGrid.appendChild(btn);
});
```

Figura 7: Implementación de la lógica de la calculadora y manejo de la pila.

5.4. Protección y Producción (Ofuscación)

Como medida de seguridad, el código de desarrollo se procesó mediante técnicas de transformación y renombrado de variables a identificadores hexadecimales e inserción de código muerto (dead code). Esto dificulta significativamente la ingeniería inversa en el entorno del cliente.

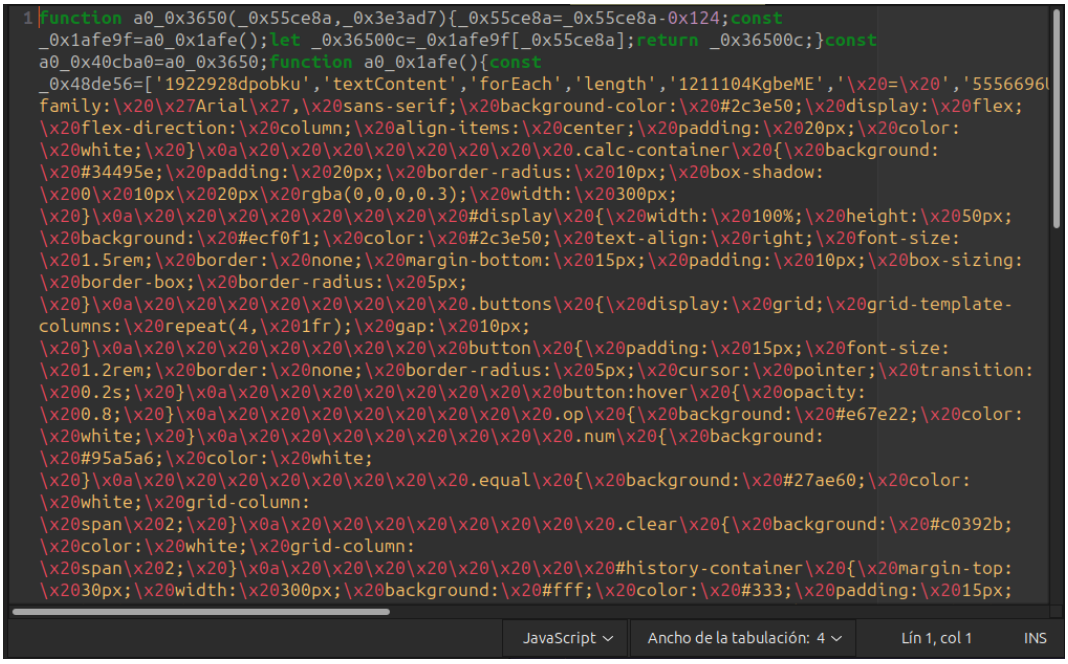

Figura 8: Captura del código ofuscado (`ejercicio02.min.js`) listo para producción.

5.5. Resultados de Ejecución

La interfaz generada muestra una calculadora funcional con un panel inferior dedicado al historial. Se verificó que al realizar múltiples operaciones, estas se apilan correctamente, mostrando la más reciente en la parte superior.

Figura 9: Interfaz dinámica y visualización de la pila de historial.

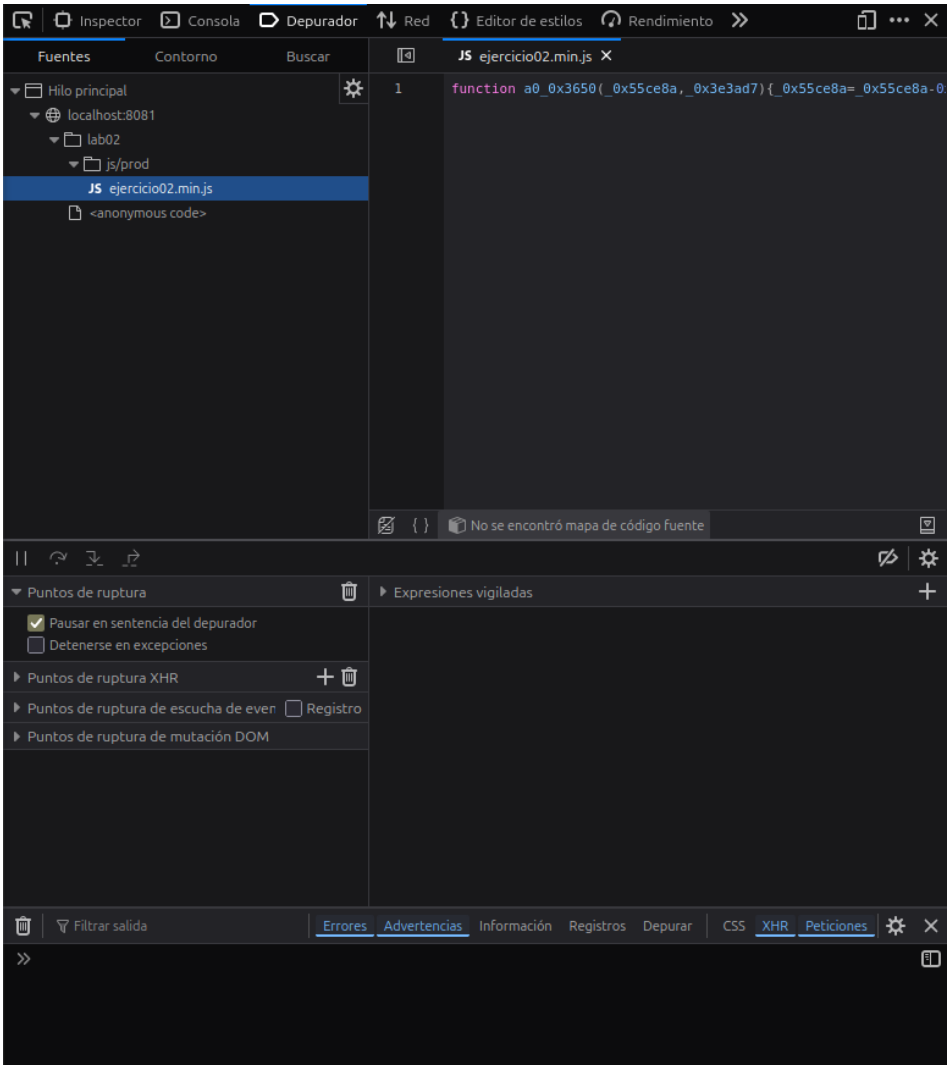

Figura 10: Verificación en el inspector del navegador mostrando la carga del script ofuscado.

6. Ejercicio 03: Juego del Ahorcado con Canvas

6.1. Descripción de la Tarea

Este ejercicio consistió en la creación de un juego interactivo del .ᴬhorcado". El desafío principal fue integrar la manipulación dinámica del DOM para la interfaz de usuario con el uso de la API de Canvas para el renderizado de gráficos en tiempo real, basándose en el estado del juego (número de errores).

6.2. Construcción Dinámica de la Interfaz

Siguiendo los requisitos del laboratorio, no se utilizó HTML estático. El contenedor principal, el lienzo de dibujo (*canvas*) y el teclado alfabético (A-Z) se generaron íntegramente mediante JavaScript.

Listing 2: Creación de elementos base y Canvas

```
const contenedorPrincipal = document.createElement("div");
document.body.appendChild(contenedorPrincipal);

const canvas = document.createElement("canvas");
canvas.width = 300;
canvas.height = 300;
const ctx = canvas.getContext("2d");
contenedorPrincipal.appendChild(canvas);
```

6.3. Lógica del Juego y Gestión de Estados

La lógica se centraliza en la función `iniciarJuego()`, la cual reinicia las variables de control, limpia el *canvas* y selecciona una palabra aleatoria de un repositorio predefinido.

- **Control de Aciertos:** Se generan *spans* vacíos que se completan dinámicamente cuando el usuario presiona la letra correcta.
- **Gestión de Errores:** Cada fallo incrementa un contador que dispara la función de dibujo y almacena la letra en un listado de "letras incorrectas"para evitar duplicidad de errores.
- **Condiciones de Finalización:** El juego evalúa tras cada interacción si el número de errores llegó al límite (10) o si todas las letras de la palabra secreta han sido descubiertas.

6.4. Renderizado Progresivo con Canvas

El dibujo del ahorcado se realiza mediante una estructura `switch` que interpreta el número de errores actuales y dibuja la parte correspondiente (base, poste, soga, cabeza, tronco y extremidades) utilizando trazos de vectores (`moveTo`, `lineTo`, `arc`).

```
// FUNCION DIBUJAR AHORCADO CANVAS
function dibujarAhorcado() {

    ctx.lineWidth = 3;

    switch (errores) {
        case 1:
            // base
            ctx.beginPath();
            ctx.moveTo(20, 280);
            ctx.lineTo(150, 280);
            ctx.stroke();
            break;

        case 2:
            // poste vertical
            ctx.beginPath();
            ctx.moveTo(70, 280);
            ctx.lineTo(70, 50);
            ctx.stroke();
            break;

        case 3:
            // poste horizontal
            ctx.beginPath();
            ctx.moveTo(70, 50);
            ctx.lineTo(180, 50);
            ctx.stroke();
            break;

        case 4:
            // cuerda
            ctx.beginPath();
            ctx.moveTo(180, 50);
            ctx.lineTo(180, 80);
            ctx.stroke();
            break;

        case 5:
            // cabeza
            ctx.beginPath();
            ctx.arc(180, 100, 20, 0, Math.PI * 2);
            ctx.stroke();
            break;
```

Figura 11: Lógica de dibujo progresivo en el Canvas según errores.

6.5. Resultados de Ejecución

Se verificó la interactividad de los botones generados y la respuesta visual inmediata del Canvas. A continuación se muestran las capturas del juego en funcionamiento, incluyendo los estados de victoria y derrota.

Figura 12: Interfaz del juego: Estado de victoria con palabra completada.

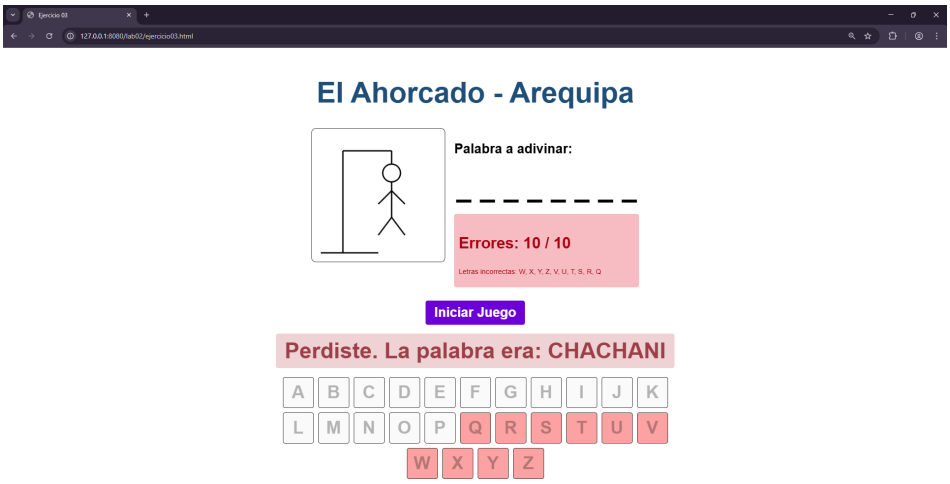

Figura 13: Interfaz del juego: Estado de derrota mostrando el dibujo completo en el Canvas.

6.6. Versión de Producción

Al igual que en los ejercicios anteriores, el archivo final `ejercicio03.min.js` fue ofuscado para garantizar que la lógica de selección de palabras y validación de intentos no sea fácilmente legible desde las herramientas de desarrollador del navegador.

```
Laboratorio 02 > js > prod > ejercicio03.min.js > ...
1        reta[_0x3b2ba3(0x109)]; _0xbf7f3f++) { letrasCorrectas['includes'](palabraSecreta[_0xbf7f3f]) && (_0xb13ae8[_0xb
```

Figura 14: Código del juego del ahorcado en modo producción.

7. Ofuscación del Código

Se generaron dos versiones:

- Desarrollo: Código legible.
- Producción: Código ofuscado.

Herramienta utilizada:

`javascript-obfuscator`

8. Despliegue con Docker

Para el despliegue se utilizó una imagen ligera basada en Nginx Alpine.

8.1. Dockerfile

```
FROM nginx:alpine

COPY . /usr/share/nginx/html/lab02

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Ventajas:

- Bajo consumo.
- Rápido arranque.
- Portabilidad.
- Fácil mantenimiento.

9. Resultados

Se verificó correctamente:

- Funcionamiento del teclado dinámico.
- Funcionamiento de la calculadora.
- Registro del historial.
- Despliegue correcto del contenedor.
- Acceso web mediante navegador.
- Carga del código ofuscado.

10. Cuestionario

1. ¿Qué ventajas tiene generar HTML desde JavaScript?
   Permite crear interfaces dinámicas, reutilizables e interactivas.
2. ¿Qué es la ofuscación?
   Es una técnica que dificulta la lectura del código fuente.
3. ¿Por qué usar Docker?
   Porque permite desplegar aplicaciones en entornos portables y reproducibles.

11. Conclusiones

1. JavaScript permite generar aplicaciones completas desde el cliente.
2. Las estructuras de datos tienen aplicación real en la web.
3. Docker simplifica despliegue y distribución.
4. La ofuscación mejora la protección del código.
5. Nginx Alpine es ideal para sitios estáticos.

12. Repositorio del Proyecto

`https://github.com/JesusFSP/Grupo-7---Laboratorio.git`

13. Video

`https://www.youtube.com/watch?v=0QKKT-QDJCg`

14. Bibliografía

- Docker Documentation. `https://docs.docker.com/`
- MDN Web Docs. `https://developer.mozilla.org/`
- Nginx Documentation. `https://nginx.org/`
- JavaScript Obfuscator. `https://obfuscator.io/`

15. Rúbricas

15.1. Entregable Informe

Cuadro 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

15.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Cuadro 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Cuadro 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4			
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	2	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4			
Total		20		12	